Quantum-Secure A2A: A Post-Quantum QUIC Transport for Agent-to-Agent Protocols

Final Year Major Project — Architecture & Implementation Guide

---

0. The one-line pitch (lead with this in your demo)

> "A2A agents today talk over TLS that quantum computers will break retroactively. We built a drop-in QUIC transport for A2A that does a hybrid classical+post-quantum handshake, hardens Kyber against side-channels, and rotates keys live — without dropping a single streaming session. Watch."

Judges remember the one sentence + the live demo, not the slide with six bullet points. Everything below builds toward that moment.

---

1. What's actually novel here (say this explicitly to judges)

Most "PQC demo" projects just swap RSA for Kyber in a TLS handshake and call it done. That's table stakes. Your differentiators, in order of how impressive they are to a judging panel:

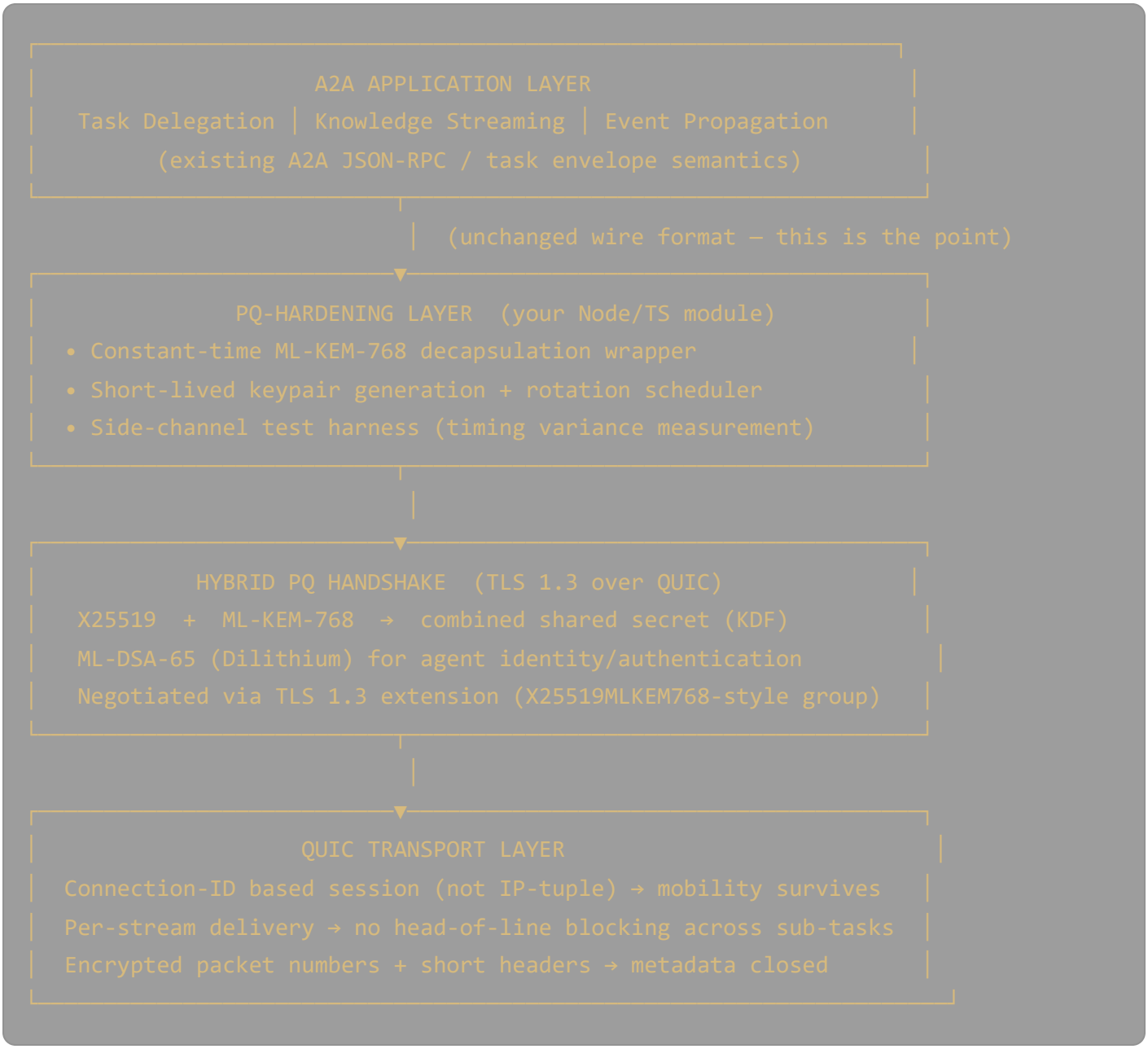
1. **Zero-downtime PQ key rotation for long-lived agent streams.** This is the genuinely hard, under-solved problem. A2A knowledge-streaming sessions can run for minutes; rotating ML-KEM keys mid-stream without breaking the QUIC connection or re-doing a full handshake is a real systems contribution, not a crypto-library wrapper.
2. **Constant-time decapsulation hardening layer as a reusable Node.js module**, independently benchmarked against timing leakage — most student projects don't measure side-channel resistance, they just assert it.
3. **A migration guide + compatibility shim** that lets an *existing* classical-TLS A2A agent talk to a PQ-upgraded agent during a transition window (version negotiation), because that's the real-world deployment problem A2A doesn't solve yet.
4. **Quantified HNDL risk model specific to agent workloads** (not generic "quantum is scary" — actual sensitivity classification of task delegation vs. knowledge streaming vs. event

propagation, with a "harvest-now decrypt-later" exposure window calculation).

Everything else (QUIC transport, hybrid handshake) is necessary scaffolding — build it solidly but don't oversell it as novel. Be upfront with judges about which parts are "engineering integration of known techniques" (QUIC+hybrid TLS) vs. "our contribution" (rotation protocol, hardening layer, migration path, HNDL model). This honesty reads *well* to technical judges.

---

2. System architecture



Why this layering matters for your report

Keep the A2A application semantics (task envelopes, JSON-RPC messages) completely untouched. Your thesis contribution is entirely in the transport + handshake + hardening layers. This is what "minimal change to application-layer agent logic" in your objectives actually means architecturally — and it's what makes the migration guide credible.

---

3. Tech stack (concrete, buildable in Node/TypeScript)

You **cannot** implement ML-KEM, ML-DSA, or a QUIC state machine from scratch in a final-year timeline — and you shouldn't; using audited primitives is literally one of your stated objectives ("audited library selection"). Your engineering work is integration, hardening, measurement, and the rotation protocol — not reimplementing cryptography.

Layer	Library / Tool	Notes
PQ + hybrid KEX primitives	**liboqs** + **oqs-provider** for OpenSSL 3.3+, or **AWS-LC / s2n-tls** (already ships ML-KEM hybrid groups)	Both are audited. s2n-tls is easier to get hybrid groups working out-of-the-box; liboqs+OpenSSL gives you more control for the hardening layer.
QUIC transport	**Node's built-in** `node:quic` (experimental, Node 22+) if stable enough, otherwise **quiche** (Cloudflare's Rust QUIC) via a Node native binding (`node-quiche` / build your own N-API wrapper), or `@fails-components/webtransport` for a WebTransport-over-QUIC abstraction	If Node's native QUIC support is too unstable in your Node version, fall back to quiche via a thin Rust↔Node FFI (napi-rs). This is a legitimate, demoable engineering decision — document it as a trade-off in your report.
PQ signatures (agent auth)	**ML-DSA-65 (Dilithium3)** via liboqs bindings (`liboqs-node`)	Used for agent identity certs, not the KEX itself.
Backend orchestration / control plane	**Node.js + Express + TypeScript**	REST/WebSocket admin API: register agents, trigger rotations, stream metrics.

Layer	Library / Tool	Notes
Agent simulation	**TypeScript agent processes** implementing the A2A task envelope over your new transport	You need ≥3 simulated agents doing task delegation, knowledge streaming, event propagation to demo all three workload profiles.
Frontend dashboard (for judges)	**Next.js 15 (App Router) + TypeScript + Tailwind + shadcn/ui**	Live visualization: handshake timeline, latency comparison (classical vs hybrid PQ), key rotation events, packet metadata comparison (TCP+TLS vs QUIC). This is what wins the room — see §7.
Real-time telemetry to dashboard	**WebSocket (ws) or Server-Sent Events** from the Node control plane	Push handshake/rotation events to the Next.js dashboard live during the demo.
Benchmarking	**autocannon / custom harness** + **hyperfine** for CLI-level timing, **perf/strace** or a cycle-counting library for side-channel timing variance	Objective #5 needs real numbers across your 3 workload profiles.
Containerization	**Docker Compose**: one container per simulated agent + control plane + dashboard	Judges love `docker compose up` producing a working multi-agent demo in one command.

Why not build QUIC/crypto from scratch

Explicitly note in your report: "We selected FIPS/NIST-validated primitives (ML-KEM-768, ML-DSA) via liboqs rather than reimplementing lattice cryptography, consistent with the project objective of audited library selection. Our contribution is the composition, hardening, rotation protocol, and hybrid handshake integration — not the primitives themselves." This is the correct, defensible framing and prevents a judge asking "did you implement Kyber yourself?" from derailing you.

---

4. The hybrid handshake — what to actually implement

1. TLS 1.3 ClientHello advertises a hybrid named group (mirror the IETF draft naming convention, e.g. `X25519MLKEM768`).
2. Client generates X25519 keypair + ML-KEM-768 keypair, sends both public keys concatenated in `key_share`.
3. Server responds with its own X25519 share + ML-KEM-768 ciphertext (KEM encapsulation).
4. Both sides derive: `combined_secret = KDF(X25519_shared_secret || ML-KEM_shared_secret)` — concatenation-then-KDF is the standard hybrid construction (matches the approach used in Chrome/BoringSSL's post-quantum TLS rollout and the IETF hybrid KEX drafts).
5. Agent authentication: certificates signed with ML-DSA-65 instead of RSA/ECDSA — validate the peer agent's identity cert chain using the PQ signature scheme.
6. This whole handshake rides inside QUIC's crypto frames (QUIC mandates TLS 1.3 for its handshake — this is literally how QUIC is specified, so you're not fighting the protocol, you're using it as designed).

---

5. Your hardest and most novel piece: live key rotation

This is where you should spend the most design + writing effort, because it's the part that isn't "just use a library."

Problem: ML-KEM keypairs should be short-lived (objective #4), but A2A knowledge-streaming sessions can be long-lived. Rotating keys naively means tearing down the QUIC connection and redoing a full handshake — which defeats QUIC's connection-migration benefit and disrupts the stream.

Your protocol (design this, prototype it, then benchmark it):

1. Define a QUIC application-layer frame (or a reserved A2A control message type) — `KEY_ROTATE_INIT` — sent by either peer on a schedule (e.g. every N minutes or after M bytes transferred).
2. Initiator generates a fresh ML-KEM-768 keypair, sends the public key in `KEY_ROTATE_INIT`.
3. Responder performs encapsulation, returns ciphertext in `KEY_ROTATE_ACK`, alongside a **short overlap window** where both the old and new derived traffic keys are valid (similar in spirit to TLS 1.3 key update, `KeyUpdate`, but carrying a fresh KEM exchange instead of just a KDF ratchet).
4. Once both sides confirm `KEY_ROTATE_ACK` receipt, old keys are wiped; in-flight packets encrypted under the old key within the overlap window are still decryptable, new packets use

the new key. This mirrors how QUIC already handles 1-RTT key phase bits — you're extending that mechanism to carry a fresh PQ KEM exchange instead of a simple ratchet.

5. **This is your benchmark story**: measure stream continuity (zero packet loss / zero re-handshake) across a rotation event during an active knowledge-streaming session, and measure rotation overhead (extra bytes, extra latency) relative to session duration.

Write this up as a small protocol spec (a few pages) in your report — sequence diagrams, state machine, failure/retry handling (what if `KEY_ROTATE_ACK` is lost?). This reads as genuine protocol design work, which is exactly what a "new kind of solution" needs to look like to judges and evaluators, versus "downloaded a PQC library."

---

6. Repository structure

```
quantum-a2a/
├── packages/
│   ├── pq-hardening/          # TS module: constant-time decap wrapper, key rotation
│   │   ├── src/
│   │   │   ├── kem.ts          # liboqs-node bindings wrapper
│   │   │   ├── rotation.ts     # key rotation protocol state machine
│   │   │   └── timing-harness.ts # side-channel measurement tooling
│   │   └── test/
│   ├── quic-transport/        # QUIC transport adapter (node:quic or quiche binding)
│   │   ├── src/
│   │   │   ├── handshake.ts    # hybrid TLS 1.3 negotiation
│   │   │   ├── connection.ts   # connection-ID based session mgmt
│   │   │   └── streams.ts      # per-stream A2A sub-task mapping
│   ├── a2a-shim/              # A2A application-layer adapter (unchanged task envel
│   ├── control-plane/         # Express + TS: agent registry, rotation triggers, me
│   └── agents/                # 3+ simulated agents: delegator, streamer, event-pub
├── apps/
│   └── dashboard/              # Next.js 15 app — live visualization for demo
├── benchmarks/                 # autocannon/hyperfine scripts + result CSVs/plots
├── docs/
│   ├── threat-model.md         # HNDL quantification (objective #1)
│   ├── rotation-protocol-spec.md
│   └── migration-guide.md      # objective: PQ migration path for existing A2A depl
├── docker-compose.yml
└── README.md
```

---

7. The Next.js judge-facing dashboard (this is what sells it)

Build a single-page live dashboard with:

1. **Handshake visualizer** — animated sequence diagram of the hybrid handshake happening in real time as an agent connects (ClientHello → hybrid key_share → ServerHello → derived secret confirmed). Judges *see* the PQ exchange happen, not just read about it.
2. **Side-by-side latency chart** — classical TLS 1.3 handshake vs. hybrid PQ handshake, across your 3 workload profiles, updating live as you run the benchmark suite.
3. **Metadata comparison panel** — a packet capture view (or simulated visualization) showing what a passive observer sees on TCP+TLS (visible IP tuple, packet lengths) vs. QUIC (encrypted packet numbers, connection-ID abstraction). This directly demonstrates your objective #2 claim.
4. **Live key rotation event log** — trigger a rotation button during a running knowledge-streaming demo session, watch the stream continue uninterrupted while a rotation event fires and completes.
5. **HNDL risk scorecard** — your threat-model quantification rendered as a simple risk matrix (workload type × sensitivity × exposure window).

Build this with Next.js App Router, a WebSocket connection to your control-plane for live event pushes, and Recharts for the latency/timing charts. Keep it visually clean — this is a systems project, not a design project, so clarity beats decoration.

---

8. Benchmarking plan (objective #5 — do this rigorously)

Workload profile	What to measure	How
Task delegation (short request/response)	Handshake latency overhead (classical vs hybrid PQ), total round-trip time	`autocannon` or custom timer harness, N=1000 runs, report p50/p95/p99
Knowledge streaming (long-lived, high-throughput)	Throughput impact of PQ overhead, rotation event impact on stream continuity	Custom TS load generator streaming synthetic "knowledge chunks", instrumented with rotation triggers mid-stream
Event propagation (fan-out, many small	Per-message overhead, head-of-line blocking comparison	`tc netem` (Linux traffic control) to simulate packet loss, measure

Workload profile	What to measure	How
messages)	(simulate packet loss, compare HTTP/2-over-TCP vs QUIC recovery time)	stream stall duration

Also benchmark: constant-time decapsulation — measure timing variance across many decapsulation calls with different secret key bit patterns, to empirically show your hardening layer reduces (not eliminates — be honest about this in your report) timing variance compared to a naive/reference implementation.

---

9. Suggested build timeline (assuming ~10-14 weeks)

1. **Weeks 1-2**: Threat model + HNDL quantification doc; environment setup (liboqs, OpenSSL 3.3+/s2n-tls, Node QUIC or quiche binding proof-of-concept).
2. **Weeks 3-4**: Get a basic hybrid TLS 1.3 handshake working standalone (script-level, not yet integrated with QUIC) — validate against known test vectors.
3. **Weeks 5-6**: Integrate hybrid handshake into QUIC transport layer; get one agent talking to another over PQ-QUIC with a trivial payload.
4. **Weeks 7-8**: Build the A2A shim layer (task envelope over your transport) + 3 simulated agents covering the 3 workload profiles.
5. **Weeks 9-10**: Design + implement the key rotation protocol; this is your riskiest, most novel component — start it early enough to have buffer.
6. **Week 11**: Constant-time hardening layer + side-channel timing harness.
7. **Week 12**: Benchmark suite across all workload profiles; collect data.
8. **Week 13**: Next.js dashboard for live demo.
9. **Week 14**: Migration guide write-up, report polish, rehearse demo.

---

10. What to say when a judge asks "isn't this just gluing libraries together?"

Have this answer ready, near-verbatim:

> "The primitives — ML-KEM, ML-DSA — are NIST-standardized and we deliberately use audited implementations rather than reimplementing lattice cryptography, which would be both infeasible in this timeframe and inadvisable for a security-critical primitive. Our contribution is

> threefold: first, a live key-rotation protocol for long-lived agent streams that has no equivalent in the current A2A spec or standard QUIC key-update mechanisms; second, an empirically-measured hardening layer with before/after timing-variance data, not just a claim of constant-time behavior; third, a concrete migration path letting classical and PQ-upgraded A2A agents interoperate during a transition window, which is an open problem the A2A spec doesn't currently address."

This framing — precise about what's novel vs. integrated — tends to land far better with technical judges than overclaiming.

---

11. Honest risks to flag in your report (judges respect this)

- Node's native QUIC support is still experimental as of recent Node releases; if you hit instability, document the fallback to a Rust (quiche) binding as a deliberate engineering decision, not a failure.
- "Constant-time" claims about your hardening layer should be *measured*, not asserted — report actual timing-variance numbers and their limitations (e.g., JIT/GC in Node make true constant-time guarantees harder than in C — call this out; it's an honest and interesting limitation to discuss, not a flaw in your work).
- Post-quantum signature and ciphertext sizes are larger than classical equivalents — quantify the bandwidth cost in your report rather than glossing over it.

Want me to go deeper on any one piece next — e.g. draft the actual `rotation.ts` state machine and protocol spec, scaffold the Next.js dashboard, or write the threat-model/HNDL quantification document first?